Efficient Heuristics for Classical Simulation of Quantum Circuits Using ZX-Calculus

Candidate no. TODO

Thesis word count: TODO

Submitted in partial completion of the
Master of Science in Advanced Computer Science

Trinity 2024

Abstract

Quantum circuits require validation and testing to ensure that they perform as expected. Classical simulation is a powerful tool for this purpose, but it is limited by the exponential growth of complexity with the number of qubits. Nevertheless, using the ZX-calculus, a graphical language for quantum mechanics, we can attempt to reduce how exponential this growth is. Recent developments have allowed focus to be placed on complexity being dependent on the number of T-gates that are in the circuit. Given t T-gates, if the complexity of the circuit is $O(2^{\alpha t})$, the aim is to reduce the value of α as much as possible.

Efficient Heuristics for Classical Simulation of Quantum Circuits Using ZX-Calculus

Candidate no. TODO

Word count: TODO

Submitted in partial completion of the

Master of Science in Advanced Computer Science

☐ I hereby certify that this is entirely my own work unless otherwise stated.

Trinity 2024

Contents

List of Figures

1 Introduction

Contents

1.1 Motivation

Quantum computing, a field rooted in the principles of quantum mechanics, represents a revolutionary approach to computation. Unlike classical computing, which relies on binary operations performed on bits, quantum computing utilizes quantum bits, or qubits, capable of representing and processing information in ways that classical bits cannot. This unique capability of qubits to exist in superpositions and entangle with each other allows quantum computers to perform certain types of calculations exponentially faster than their classical counterparts. If a classical computer were limited by memory and runtime, a quantum computer could potentially bypass these scaling issues - dubbed the "quantum advantage".

If scalable quantum computers were to become a reality, they could solve some of the most challenging problems in computer science much more efficiently than classical computers. One notable example is Shor's algorithm for factoring large numbers, which could break widely-used cryptographic systems such as RSA. Another significant algorithm is Grover's search algorithm, which provides quadratic speed-ups for unstructured search problems.

Classical computations play an essential role in assisting and accelerating the development of quantum computers. Classical simulation is a fundamental aspect of understanding and designing quantum hardware, often serving as the only means to validate these quantum systems [1]. Additionally, quantum algorithms can be implemented on classical simulators for testing and verification purposes while full-fledged quantum computers remain beyond reach (for now). These simulators emulate the behavior of quantum computers, allowing researchers to simulate processes as if running on actual quantum devices.

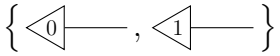
This thesis aims to present the speedup of classical simulation of quantum circuits in an almost entirely graphical approach. From the introduction of the qubits, to the representation of quantum circuits and linear maps, the underlying linear algebra that governs the operations of quantum computing is represented in a graphical manner. Using ZX-Calculus, a graphical calculus detailed in section 1.3, quantum circuits can be represented as diagrams, allowing for different discoveries to be more easily made when analysing the structure of said diagrams.

1.2 Basic Notation

In classical computing, a bit is the fundamental unit of information, and is denoted by a boolean value $b \in \{0, 1\}$. In quantum computing, the fundamental unit of information is the qubit, which we denote as $|b\rangle$, where $b \in \{0, 1\}$. Diagramatically, they are represented as follows:

We consider these the computational or Z-basis states.

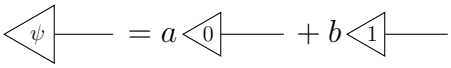
Definition 1.2.1 (Z-basis)**.** The *Z-basis* or computational basis is defined as



▲

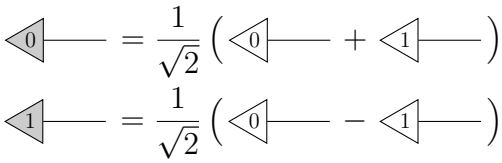
So far, the expressive power is still similar to classical computing. However, the power of quantum computing comes from the ability to exist in superpositions of these states. Concretely, the qubit can be represented in a linear combination of the basis states.

Definition 1.2.2 (Pure state)**.** A *pure state* $|\psi\rangle$ is a state



where $a, b \in \mathbb{C}$ and $|a|^2 + |b|^2 = 1$. ▲

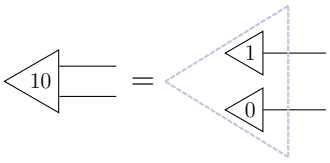
Then, the '+' or X-basis states can be defined.



(1.1)

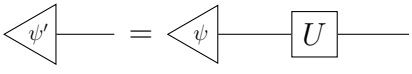
Along with the Z-basis states, these states form the building blocks of understanding quantum computing and the use of ZX-calculus for diagrammatic reasoning.

Intuitively, using the diagrammatic representation, we can consider combining diagrams in parallel. In the literature, this is the same as taking a *tensor product*. For example, we can have a 2-qubit state.



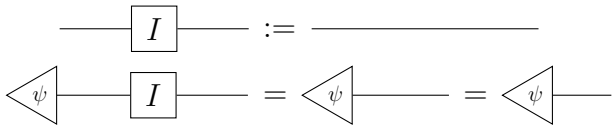
(1.2)

States can be transformed by quantum gates simply by *composing* them to obtain a new state. Given that composing the quantum gate U to a state $|\psi\rangle$ gives a state $|\psi'\rangle$, we can just connect the wires.



(1.3)

Intuitively, this means that the wire by itself should just be the identity gate, since composing it with any state should not change that state.

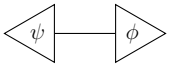


(1.4)

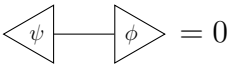
The dual of states are the effects. Here, we have the Z-basis effects.

For a state $|\psi\rangle$ and an effect $\langle\phi|$, we can obtain a scalar inner product. This is essentially the *bra-ket* notation in diagrammatic form by just connecting the wires, which comes with a few other definitions.

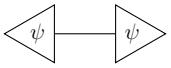
Definition 1.2.3 (Inner product [2])**.** The *inner product* $\langle\phi|\psi\rangle$ of two states $|\psi\rangle$ and $|\phi\rangle$ is defined as



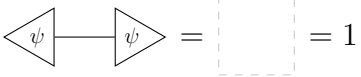
They are *orthogonal* if



The *squared-norm* of a state $|\psi\rangle$ is



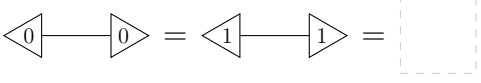
$|\psi\rangle$ is *normalised* if

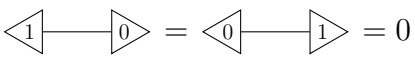


where the empty diagram represents the scalar 1. ▲

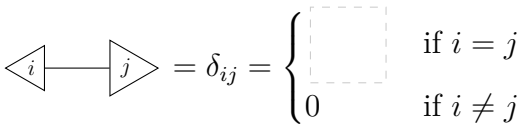
We can then define the inner product of the Z-basis states.

Definition 1.2.4. The inner product of the Z-basis states is





Succinctly, for $i, j \in \{0, 1\}$,

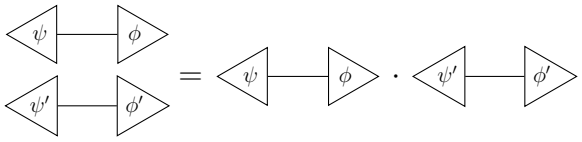


where δ_{ij} is the Kronecker delta. ▲

In fact, the succinct definition can be used to determine if a basis is an *orthonormal basis* (ONB). The X-basis is also an ONB.

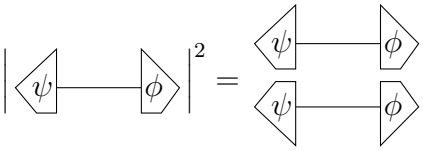
We are almost at the stage where we can obtain probabilities from the diagrams. The scalar inner product is not really a probability, that is, a number between 0 and 1, but instead, a number $z \in \mathbb{C}$.

First, we note that putting multiple scalars in a single diagram is just the same as multiplying them together.



(1.5)

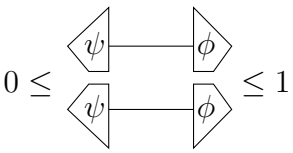
We also know that for a complex number and its conjugate, $z, \bar{z} \in \mathbb{C}, |z|^2 = z\bar{z}$ Translating that to the diagrammatic representation, we can obtain the following.



(1.6)

Note that assymetry is introduced to denote the conjugation. Remember that a complex number $z = a + bi \in \mathbb{C}$ has a conjugate $\bar{z} = a - bi \in \mathbb{C}$, so this corresponds to a vertical reflection in the diagram above.

Using the *Born rule*, if we make sure that $|\psi\rangle$ and $|\phi\rangle$ are normalised, we can obtain the following.



(1.7)

This can be interpreted as our desired probaility - the probability of measuring $|\psi\rangle$ in the state $|\phi\rangle$.

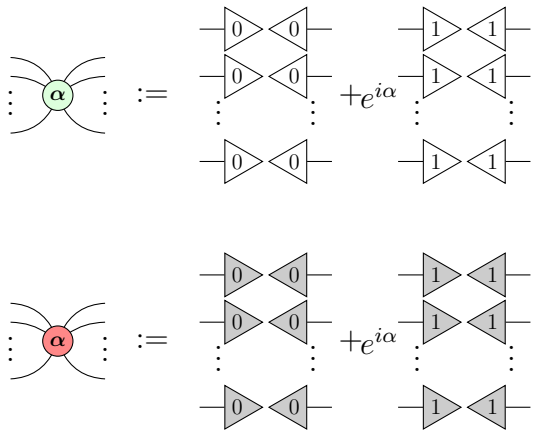
We now have all we need to introduce the ZX-calculus notation.

1.3 ZX-Calculus

So far, we have introduced the representation of quantum states, effects, and inner products in a diagrammatic form. However, there still isn't much we can do without a way to manipulate them. In the previous section, we mentioned that we can apply quantum gates to states to obtain new states. Here, we will introduce the ZX-Calculus and show the highly intuitive representation of some quantum gates such as the Hadamard and CNOT gates, based on the building blocks from the previous section. The ZX-Calculus is a graphical calculus for quantum computations, developed by Coecke and Duncan in 2008 [3]. All operations are expressed as *spiders*, connected by *edges* (or *wires*).

1.3.1 Spiders

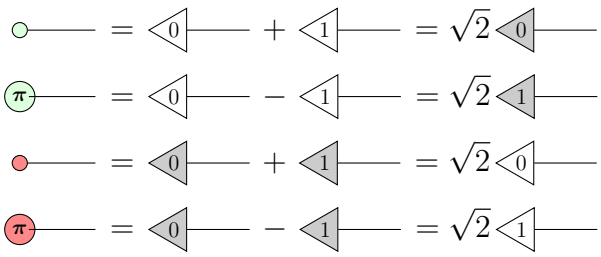
Definition 1.3.1 (Spiders)**.**



▲

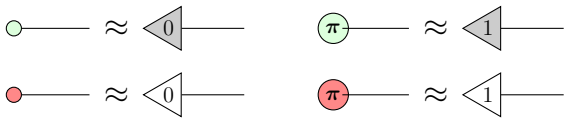
Here, we have green and red spiders, also known as Z and X spiders, made of the Z and X basis states respectively. Wires coming in from the left are *inputs*, and wires going out to the right are *outputs*. The α inside a spider node is also called the *phase* of the spider. For example, with $\alpha = 0, e^{i\alpha} =$. It is convenient to denote spiders with $\alpha = 0$ as having no angle written inside the spider.

Immediately, we can see that the Z and X basis states can be formed from the definition of the spiders, up to the multiplicative constant $\sqrt{2}$. Note that $e^{i\pi} = -1$.



(1.8)

It turns out that in reasoning with spiders, we can ignore the multiplicative constants, since much of the manipulation of the diagrams is based on the structure of the diagrams themselves. The scalars are easy to determine using the definitions we have seen so far. By using $\approx$ to denote equality up to a non-zero factor, we have

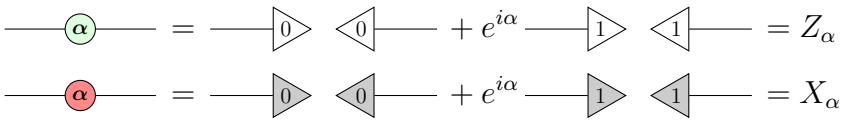


(1.9)

Scalars can also be represented using just spiders.

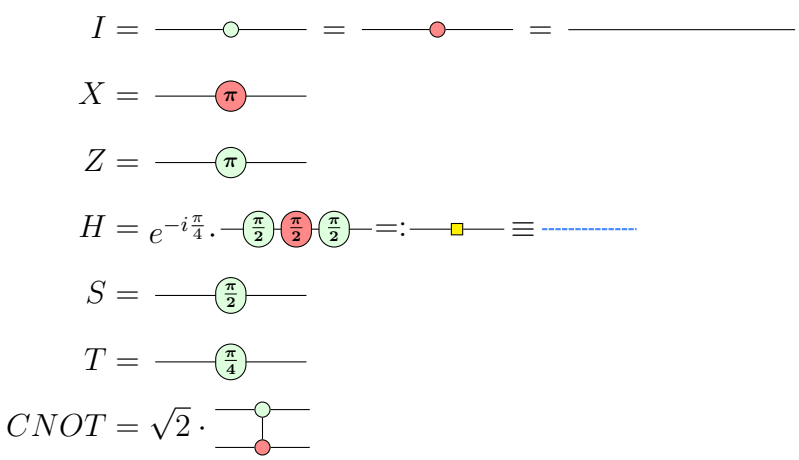
(1.10)

We can also see the single-qubit gates in the ZX-Calculus.



(1.11)

We can then define many of the common quantum gates in this way.



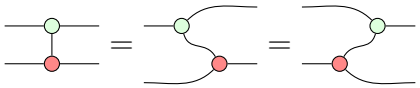
(1.12)

There are 2 gates of special note here. The Hadamard gate H, denoted by the small yellow box, or as a blue dashed line, is a single-qubit gate that interchanges

Z and X bases. We will see later in the ZX-rules that this is vital for the colour change (***cc***) rule, as well as why it is useful to denote it as the blue dashed line.

The CNOT gate is a two-qubit gate that acts on the target qubit if the control qubit is in the state $|1\rangle \approx$

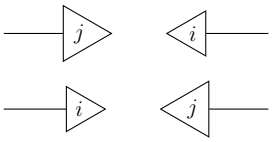
. The vertical line is used simply because it does not matter which direction the line is going, where the following equality of the second and third forms can be verified.



(1.13)

Last but not least, we also introduce the SWAP gate, which as the name suggests, swaps two qubits.

$$SWAP = \quad := \sum_{i,j\in\{0,1\}}$$



(1.14)

We can now introduce the ZX-rules in the next section.

1.3.2 ZX-Rules

The ZX-Calculus is based on a set of rules that allow us to manipulate the diagrams. For $\alpha, \beta \in [0, 2\pi)$ and $a \in \{0, 1\}$, the rules are as follows.

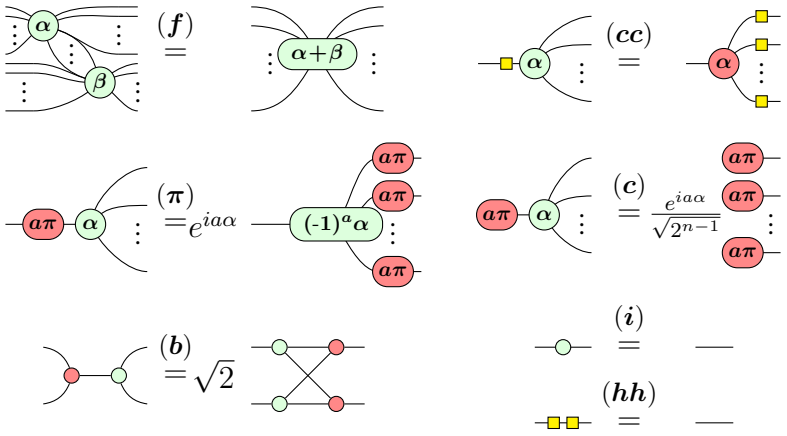


Figure 1.1: ZX-rules presented in [4]

The names of the rules are spider (***f***)usion, colour change (***cc***) rule, (***π***)-commutation, (***c***)opy rule, (***b***)ialgebra, (***i***)dentity rule, and (***hh***)adamard cancel

rule. For (***c***), n refers to the number of outputs. By duality, the rules also hold if the colours are swapped.

The power of ZX-diagrams comes from the fact that two diagrams are equivalent as long as the connectivity of the diagram is the same - that is, as long as order of inputs and outputs of the whole diagram is preserved. This does not depend on the position of the spiders or the direction of wires between them. This neat feature of ZX-diagrams is called "Only Connectivity Matters" (OCM).

We can then obtain caps and cups using 2-legged spiders together with (***i***).

(1.15)

Caps and cups obey the *yanking equations*.

(1.16)

Before we move on to reasoning about the diagrams in a graph-theoretic manner, we first introduce the concept of a *Clifford+T diagram*.

Definition 1.3.2 (Clifford diagram)**.** A *Clifford diagram* is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{2}$. ▲

Definition 1.3.3 (Clifford+T diagram)**.** A *Clifford+T diagram* is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{4}$. ▲

It is clear from the definitions that a Clifford diagram is a subset of a Clifford+T diagram. The reason we introduce these concepts is that in trying to obtain our main goal of the thesis, it is sufficient to reason about Clifford+T diagrams, as a result of the following theorem.

Theorem 1.3.4 (Approximate Universality [5])**.** Clifford+T diagrams are *approximately universal* for quantum computing - they can approximate any quantum circuit.

The proof of the above theorem involves the *Solovay-Kitaev theorem*, combining with the fact that any single-qubit phase gate can be approximated by a sequence of Hadamard and T gates, with details found in [5].

1.4 Graph Reduction

Because of OCM, ZX-diagrams up until now have very similar structure to graphs, where the spiders are the vertices and the wires are the edges. The only difference may be that the spiders are coloured, and that the Hadamard boxes are included. In this section, we solidify the connection between ZX-diagrams and graphs, and introduce an algorithm that will simplify ZX-diagrams, defining what it means for a graph to be simplified. Ultimately, simplifying ZX-diagrams should allow us to reduce the number of spiders in the diagram, which directly translates to a reduction in the number of gates in the quantum circuit. In turn, as we will see in the next chapter, this will lead to a speedup in the simulation of quantum circuits.

First, we define what it means for ZX-diagram to be *graph-like*.

Definition 1.4.1 (Graph-like [6])**.** A ZX-diagram is *graph-like* when

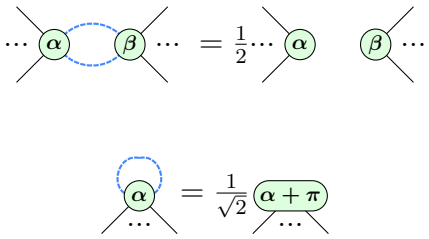
- Every spider is a Z-spider.
- Spiders are only connected via Hadamard edges.
- There is at most only one edge between each pair of spiders.
- There are no self-loops.
- Every input and output wire is connected to a Z-spider.

▲

Just from this definition, Hadamard edges are in fact the only edges in the graph-like ZX-diagram, leading to the convenience of having them as the blue dashed lines introduced earlier in (1.12). The following proposition can then be proven, using rewrite rules introduced previously in ZX-Rules.

Proposition 1.4.2 ([6])**.** Every ZX-diagram can be converted to a graph-like ZX-diagram in polynomial time.

The proof [6] makes use of the fact that Hadamard edges in parallel cancel, and that we can always remove a Hadamard self-loop. Parallel edges cancelling is an application of the *Hopf rule*, derivable from the ZX-rules.



(1.17)

We can simplify the graph-like ZX-diagram further. Now, the following two rewrite rules, derivable from the original ZX-rules, are introduced.

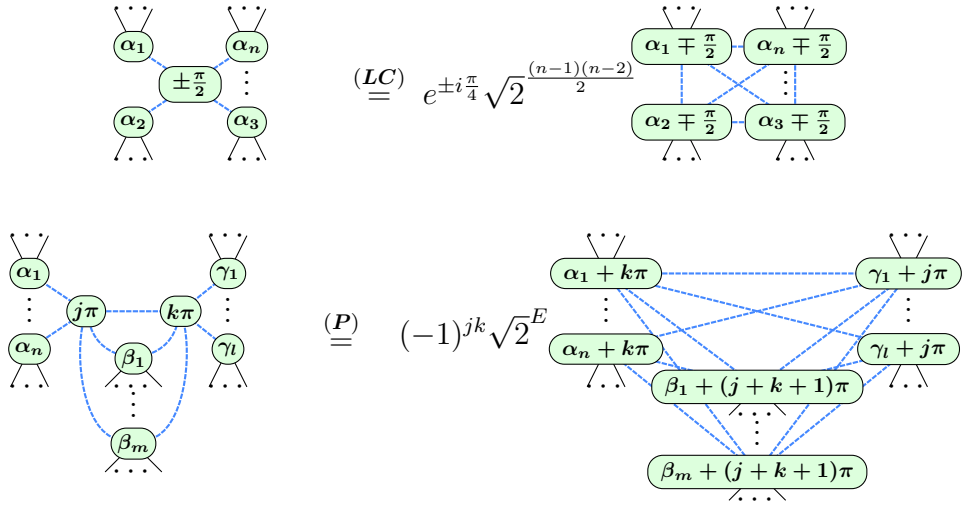


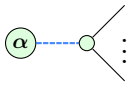
Figure 1.2: Derived ZX-rules [7], where $\alpha, \beta, \gamma \in [0, 2\pi)$ and $j, k \in \{0, 1\}$.

The rules are *local complementation* (***LC***) and *pivoting* (***P***) respectively. Here, $E = (n-1)m + (l-1)m + (n-1)(l-1)$.

Repeatedly simplifying and reducing diagrams in this manner will end up giving us patterns in the diagrams. The following definitions help us identify these patterns. First, we define interior and boundary spiders, and then phase gadgets.

Definition 1.4.3 ([8])**.** A *boundary spider* is a spider connected to an input or output. All other spiders are *internal spiders*. ▲

Definition 1.4.4 (Phase gadget [8])**.** A *phase gadget* is an arity-1 spider with angle α, connected via a Hadamard edge to a spider with no angle.



▲

These help us define the reduced gadget form.

Definition 1.4.5 (Reduced gadget form [8])**.** A graph-like ZX-diagram is in *reduced gadget form* when

- Every internal spider is a non-Clifford spider or part of a non-Clifford phase-gadget.
- Every phase-gadget has more than one target.
- No two phase-gadgets have the same set of targets.

▲

Example 1.4.6. The following diagram from [9] is in reduced gadget form. ▲

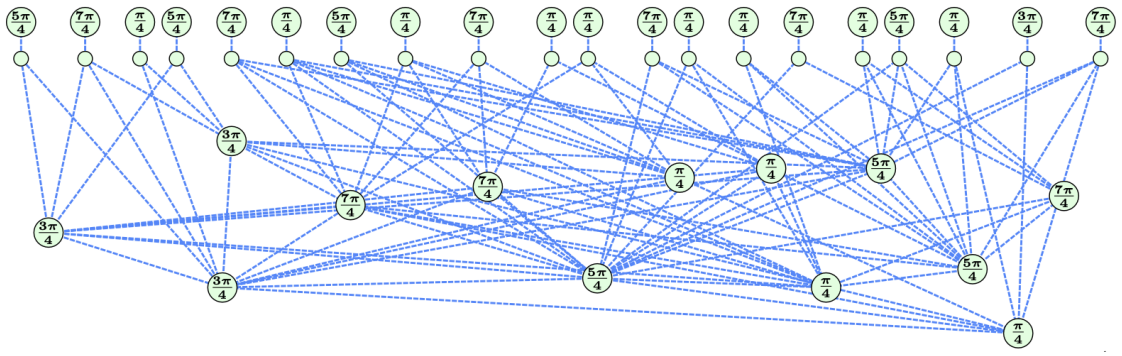


Figure 1.3: A ZX-diagram in reduced gadget form [9]

It turns out that there is an algorithm that is bounded in $O(n^3)$ where n is the number of spiders, to convert any graph-like ZX-diagram to reduced gadget form [9]. Thus, much of the work in this thesis will focus on the reduced gadget form, as it tends to bring us closer to the goal of reducing overall T-count. However, there are exceptions to this, as we will see in the next chapter.

2
Classical Simulation

Contents

2.1 Introduction

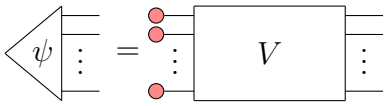
As mentioned in the previous chapter, classical simulation of quantum circuits provides an essential tool for verification and validation, despite its inherent limitations. This chapter delves into the methods and advancements in classical simulation, particularly focusing on stabiliser decomposition and the role of magic states in extending the capabilities of these simulations. Classical simulation methods that store a complete description of an n-qubit quantum state as a complex vector of size 2^n are effective only for a small number of qubits, typically around 30. For example, Shor's factoring algorithm has been simulated with 31 qubits and approximately half a million gates [10]. However, for certain classes of quantum circuits, such as those that can be represented in a Clifford diagram,

classical simulation can be significantly more efficient [11]. This is known as the *Gottesman-Knill theorem*.

Theorem 2.1.1 (Gottesman-Knill Theorem)**.** A Clifford computation can be efficiently classically simulated.

In other words, *Clifford diagrams* are efficiently classically simulated. ZX-Calculus not only aids in visualizing quantum operations but also simplifies the process of identifying and manipulating *stabiliser states*.

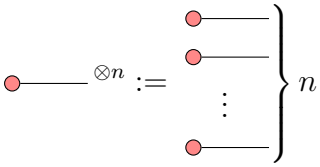
Definition 2.1.2 (Stabiliser state [11, 12])**.** An n-qubit state $|\psi\rangle$ is a *stabiliser state* if it has the form



where V is a Clifford diagram. ▲

Stabiliser circuits are particularly notable for their applications in quantum error correction [11]. However, when considering Clifford+T circuits, we need to introduce *magic states*. As we see in this chapter, these then allow us to decompose the circuit into a sum of stabiliser states, which are all efficiently classically simulatable, with the cost of having an exponential number of them.

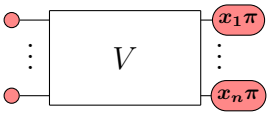
WLOG, when simulating quantum circuits, we can assume that the input state to our n-qubit circuits is of the form



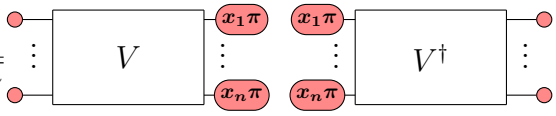
(2.1)

2.2 Strong Simulation

Given that we have a n-qubit circuit with the input state from (2.1), we would then want to obtain the probability of measuring a certain output state. For instance, we can plug in the outputs as spiders as such.



Then using the Born rule, we can obtain the probability of measuring the output state [6].

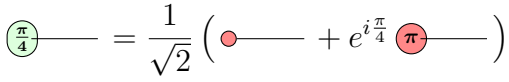
$$P(x_1, x_2, ..., x_n) = \frac{1}{\sqrt{2^{4n}}}$$  (2.2)

$V^\dagger$ is used to denote the adjoint of V, which simply means the inputs and outputs are swapped, and the spider phases are negated. The scalar factors of $1/\sqrt{2}$ comes from those seen from Eq. 1.8. If V is a Clifford diagram, then following from the Gottesman-Knill Theorem, the scalars can be computed efficiently. Obtaining the probabilities this way is known as *strong simulation*.

A natural question that arises is whether the same can be done if V is a Clifford+T diagram instead. The following sections will delve into the methods used to simulate such circuits.

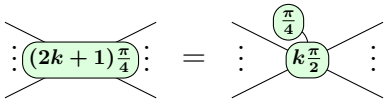
2.3 Stabiliser Decomposition

Definition 2.3.1 (Magic state [12])**.** The *magic state* is



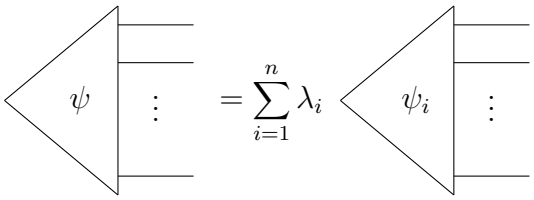
▲

The naïve approach to make use of this magic state decomposition is to then apply it to every single T-like spider in the Clifford+T diagram, where we define a T-like spider as a spider with a phase of $(2k+1)\frac{\pi}{4}$ for some $k \in \mathbb{Z}$. By un($\boldsymbol{f}$)using the magic state from these spiders as below, we can apply the Magic state [12] decomposition to obtain a sum of stabiliser states.

 (2.3)

This sum can be called the *stabiliser decomposition* of the circuit.

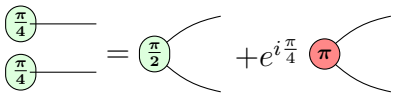
Definition 2.3.2 (Stabiliser Decomposition [12])**.** The *stabiliser decomposition* of a state $|\psi\rangle$ is a sum of stabiliser states



where $\lambda_i \in \mathbb{C}$ and $|\psi_i\rangle$ are stabiliser states. ▲

Using the Magic state [12] decomposition, we can see that the value of $n = O(2^t)$ if we apply it to the t T-like spiders in the circuit. We can call t the *T-count* of the circuit.

However, there is an even better decomposition



(2.4)

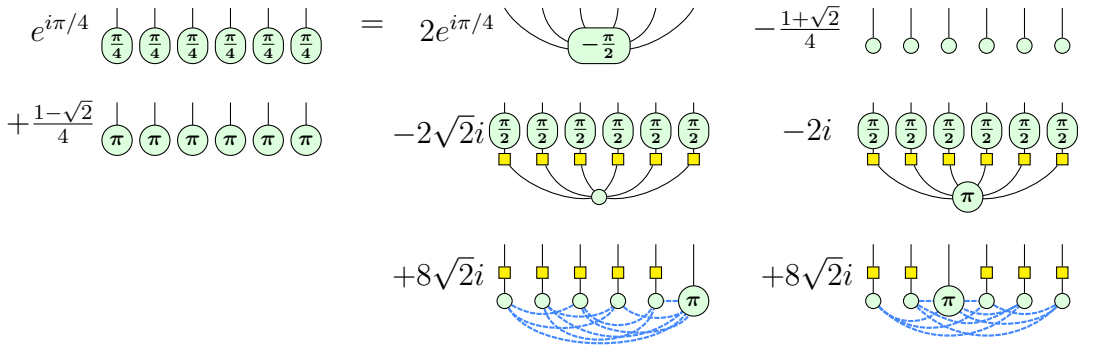
This allows us to decompose the circuit into a sum of stabiliser states with $n = O(2^{\frac{t}{2}})$. Different methods of decomposing the circuit can lead to different values of n. Since n is believed to be exponential in t (or else we have $P = NP$) [13], we can take it that $n = O(2^{\alpha t})$ for some $0 < \alpha < 1$. We can then compare the efficiency of different methods by using this metric.

Definition 2.3.3 ((In)efficiency [9, 12, 14, 15])**.** The *(in)efficiency* of a stabiliser decomposition that reduces the T-count by r using p stabiliser states is

$$\alpha = \frac{\log_2(p)}{r}$$

where a lower value of α indicates a more efficient decomposition. ▲

For example, the Bravyi-Smith-Smolin (BSS) decomposition [12] makes use of the following decomposition which has $\alpha \approx 0.468$.

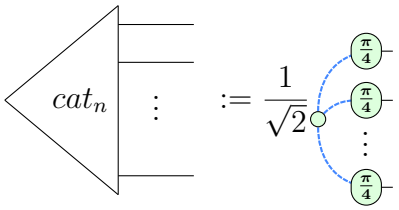


(2.5)

Using this metric, we can even compare the efficiency of different algorithms used to decompose the circuit. It also allows us to obtain the efficiency of different decompositions which do not use magic states at all. As we will see in the next few chapters, this ties in well when we consider the higher efficiency of *cat state* decomposition.

2.4 Cat States

Definition 2.4.1 (Cat state [16])**.** A *cat state* is defined as

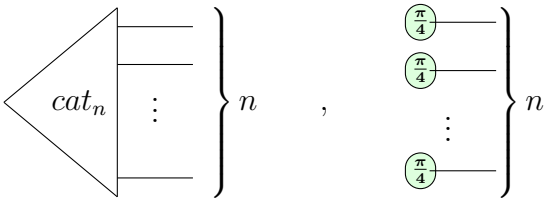


(2.6)

▲

Cat states are used because their decompositions have low value of α, as well as the following proposition, which bounds the α of the corresponding magic state decomposition.

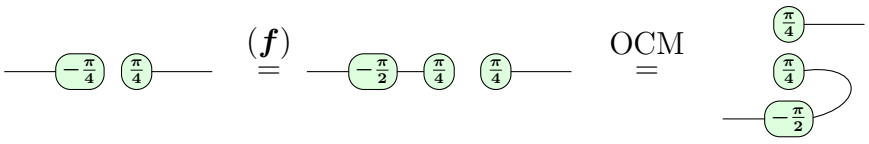
Proposition 2.4.2. Suppose α_1, α_2 are the (in)efficiency metrics of the decompositions of



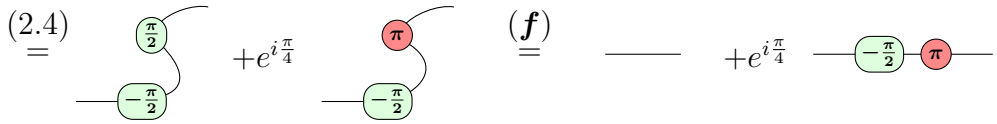
respectively. Then

$$\alpha_1 \leq r \implies \alpha_2 \leq r + \frac{1}{n}$$

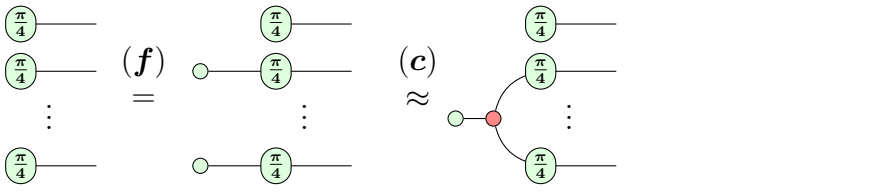
Proof. The proof is adapted from [16]. Suppose $\alpha_1 \leq r$. First, we show the following equality.



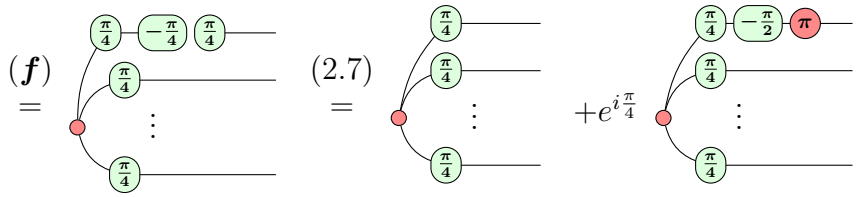
(2.7)



Then we can show the following.



(2.8)

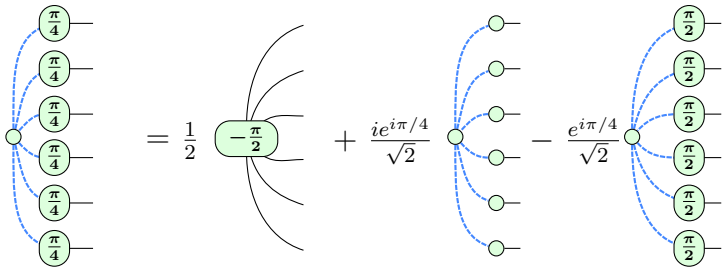


There are 2 copies of $|\text{cat}_n\rangle$ which would each give at most $2^{n\alpha_1}$ stabiliser states, for a total of at most $2^{n\alpha_1+1}$ stabiliser states. Thus, we have

$$\alpha_2 \leq \frac{n\alpha_1 + 1}{n} \leq r + \frac{1}{n}$$

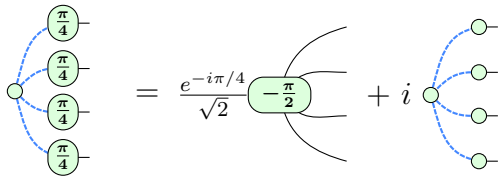
□

There are various cat state decompositions that have been found. For example, the decomposition for $|\text{cat}_6\rangle$ is



(2.9)

It has an efficiency of $\alpha \approx 0.264$. Even better, for $|\text{cat}_4\rangle$, we have the decomposition



(2.10)

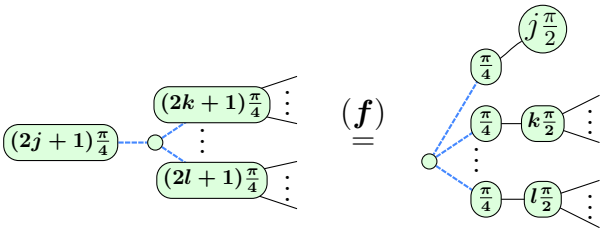
which has an efficiency of $\alpha = 0.25$.

Table 2.1 shows the best known efficiencies of the $|\text{cat}_n\rangle$ decomposition for different values of n [14].

n	3	4	5	6	7	8	9	10	11	12	13	14
terms	2	2	3	3	6	6	9	9	27	27	27	27
$\sim \alpha$	0.333	0.25	0.317	0.264	0.369	0.323	0.352	0.317	0.432	0.396	0.366	0.340

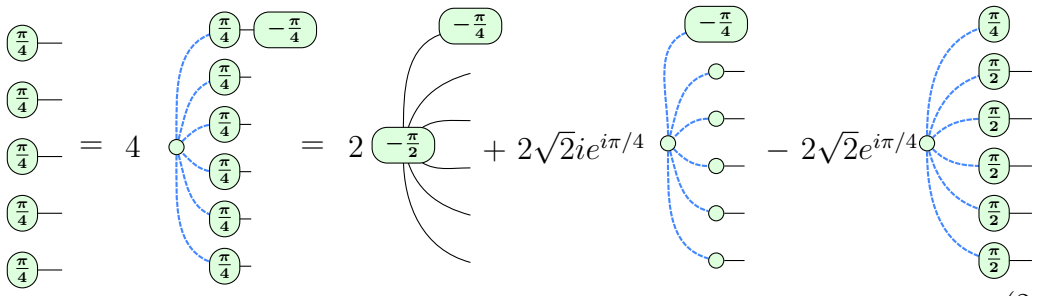
Table 2.1: $|\text{cat}_n\rangle$ decomposition

Recall that if we are dealing with the Reduced gadget form [8], any internal spider that is Clifford will be part of a non-Clifford phase gadget, by definition. Further, no two Clifford spiders will be neighbours [15]. Then, we can apply the cat state decompositions to the cat states centered at the 0-spiders that are part of these phase gadgets in the reduced gadget form, if they have ≥ 3 neighbours. This can be seen below, where $j, k, l \in \mathbb{Z}$.



(2.11)

However, if the cat states are not present, we would need to fallback to a magic state decomposition. Here, the discovery of the (non-stabiliser) decomposition of 5-qubit magic states using the decomposition of $|\text{cat}_6\rangle$ gives us a guaranteed way to decompose a circuit with a T-count ≥ 5 [15].



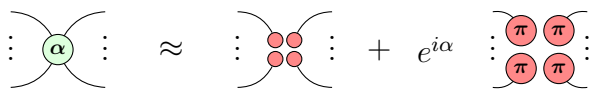
(2.12)

We then have a resulting $\alpha = \frac{\log_2(3)}{4} \approx 0.396$, which is already better than the BSS decomposition. In the next chapter, we will continue using the algorithm from [15] and improve upon the use of these cat and magic state decompositions.

2.5 Graph Cuts

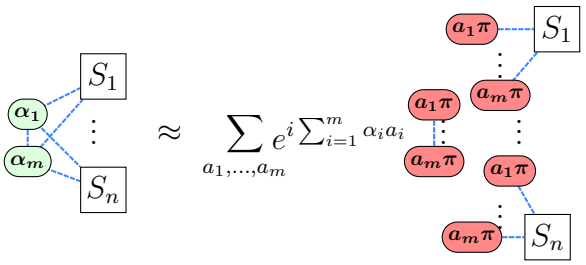
In a complementary approach to reducing the effective α of the stabiliser decomposition, graph cuts offer a different perspective [17]. Suppose we have 2 ZX-diagrams S_1, S_2 of T-counts t_1, t_2 that both have a stabiliser decomposition (in)efficiency of α respectively. If we combine the diagrams into a single diagram S, we could expect the stabiliser decomposition (in)efficiency of S to be α, coming from having $2^{\alpha(t_1+t_2)}$ stabiliser states. However, since we can treat S_1, S_2 independently and multiply their stabiliser decompositions, the actual number of stabiliser states is instead $2^{\alpha t_1} + 2^{\alpha t_2} \leq 2^{\alpha t+1}$, where $t = \max(t_1, t_2)$. This gives a resulting $\alpha' \leq \alpha\frac{t+1}{t_1+t_2}$. In the ideal case, we have that $t_1 = t_2$, so $\alpha' \leq \frac{\alpha}{2} + \frac{1}{t_1+t_2}$, a vast improvement especially for a high T-count. It is with this motivation that graph cuts are another promising approach.

First, by definition, we can see that



(2.13)

Then if we have a diagram with subdiagrams $S_1, ..., S_n$, we can see that the following holds.



(2.14)

At the cost of m cuts, and thus 2^m terms, the diagram is split into $\leq m^2 + n$ subdiagrams that can be treated independently. If α_i are all T-like, then the $\leq m^2$ scalars formed from $a_i\pi$-spiders connected to each other have T-counts of 0, and there are only $\leq n$ subdiagrams with T-counts > 0. If each subdiagram S_i has the T-count of t_i, we have a sum of 2^m terms, each value which can be obtained with just $\leq m^2 + \sum_{i=1}^n 2^{\alpha t_i}$ stabilizer terms. This gives a total of $\leq 2^m(m^2 + \sum_{i=1}^n 2^{\alpha t_i})$ stabilizer terms. We can obtain the efficiency.

$$\begin{aligned}\alpha' &\leq \frac{\log_2(2^m(m^2 + n\cdot 2^{\alpha t}))}{\sum_{i=1}^n t_i} \\ &= \frac{m + \log_2(m^2 + n\cdot 2^{\alpha t})}{\sum_{i=1}^n t_i} \\ &\leq \frac{m + 2\log_2(m) + \sum_{i=1}^n \alpha t_i}{\sum_{i=1}^n t_i}\end{aligned} \tag{2.15}$$

In the ideal case where $t_i = t$ for all i, we have $\alpha' \leq \frac{\alpha}{n} + \frac{m+\log_2(m^2 n)}{nt}$.

Another way graph cuts also reduce the effective α need not necessarily be by obtaining disconnected subdiagrams. On pseudo-structured circuits produced from CNOTs, phase gates, and Toffolis, graph cuts were used to take advantage of the structure to optimise for the use of spider ($\boldsymbol{f}$)usion in cascading fashion, where a single cut can lead to a large reduction in the T-count [18]. There, the effective efficiency obtained was $0.1 \lesssim \alpha \lesssim 0.2$.

The same was also found for extremely structured circuits involving cat states, where a single decomposition reduced the T-count by 286, for an $\alpha \approx 0.0035$ [17].

2.6 Subgraph Complement

A further addition to the strength of graph cuts is the subgraph complement approach. For a graph-like ZX-diagram (V, E), with K being the set of edges for

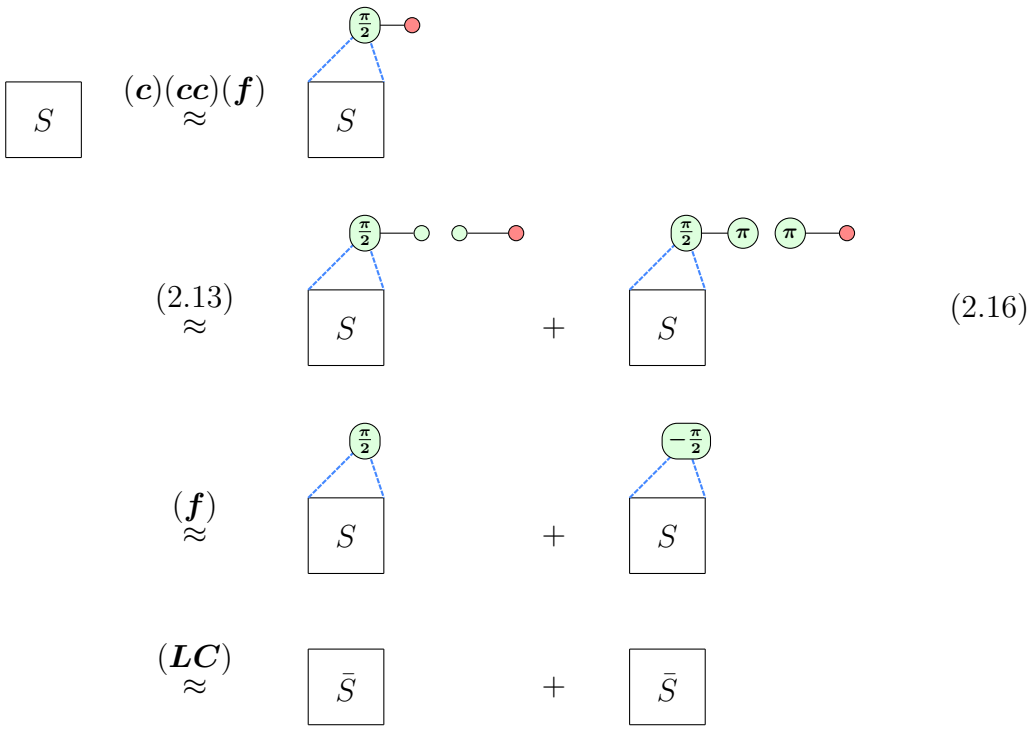
a complete graph on V, then its complement is simply $(V, K \backslash E)$. If we analyse $(\boldsymbol{LC})$ closely, and consider Hopf rule making parallel edges cancel, there seems to be a way to obtain a subgraph complement with some manipulation of ZX-rules. In fact, we have the following result.

Proposition 2.6.1 ([17])**.** Suppose we have a subdiagram S of a graph-like ZX-diagram.

$$\boxed{S} \approx \boxed{\bar{S}} + \boxed{\bar{S}}$$

where $\bar{S}$ is the complement of S.

Proof. The proof is from [17].

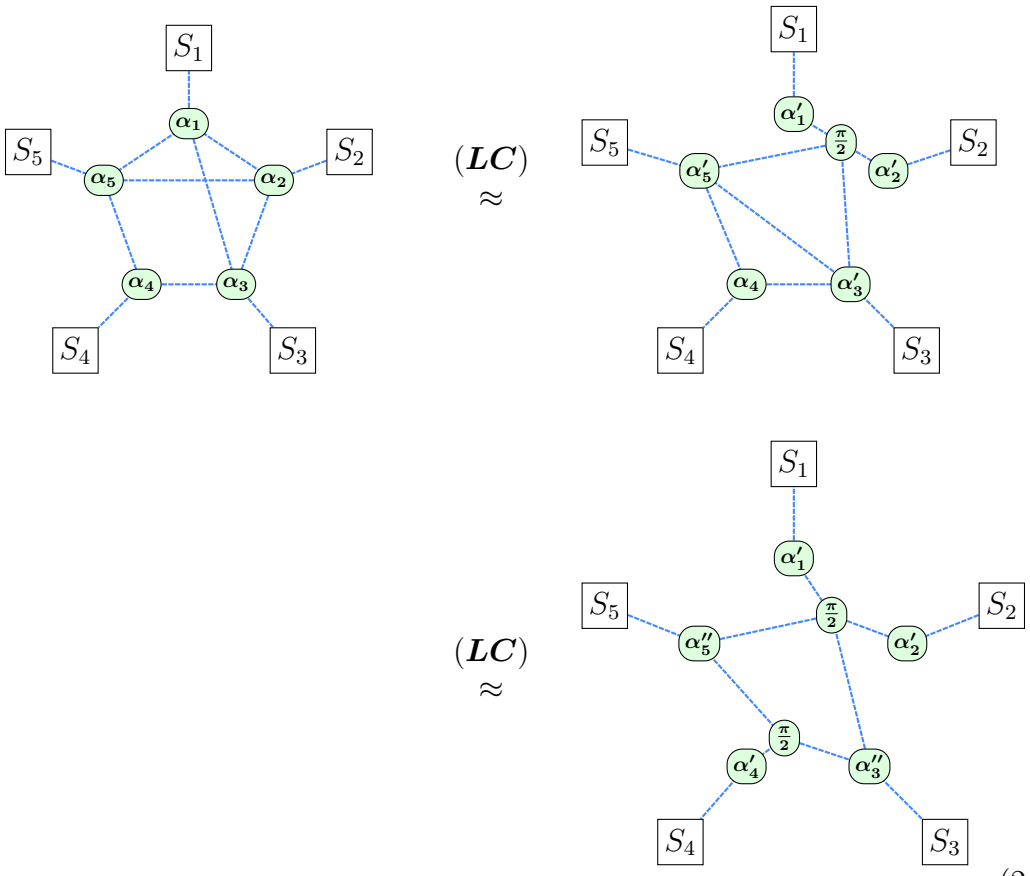


□

The power of this proposition can be demonstrated with the following example. Suppose we have the following graph-like ZX-diagram, where S_i are subdiagrams.

Following from Equation 2.14, we can apply 5 cuts to split the diagram into 5 disconnected subdiagrams, giving us 2^5 terms.

However, we can apply the subgraph complement proposition.



(2.17)

Now, we only need 2 cuts to split the diagram into the same 5 disconnected subdiagrams, giving us 2^2 terms.

Appendices

References

[1] Xiaosi Xu et al. *A Herculean task: Classical simulation of quantum computers*. 2023. arXiv: 2302.08880.

[2] Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes*. Cambridge University Press, 2017. URL: https://books.google.com.sa/books?id=I9gcDgAAQBAJ.

[3] Bob Coecke and Ross Duncan. "Interacting quantum observables: categorical algebra and diagrammatics". In: *New Journal of Physics* 13.4 (Apr. 2011), p. 043016. URL: http://dx.doi.org/10.1088/1367-2630/13/4/043016.

[4] Aleks Kissinger and John van de Wetering. "Universal MBQC with generalised parity-phase interactions and Pauli measurements". In: *Quantum* 3 (Apr. 2019), p. 134. URL: http://dx.doi.org/10.22331/q-2019-04-26-134.

[5] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[6] Aleks Kissinger and John van de Wetering. *Picturing Quantum Software*. 2024.

[7] Ross Duncan et al. "Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus". In: *Quantum* 4 (June 2020), p. 279. URL: https://doi.org/10.22331/q-2020-06-04-279.

[8] Aleks Kissinger and John van de Wetering. "Reducing the number of non-Clifford gates in quantum circuits". In: *Phys. Rev. A* 102 (2 Aug. 2020), p. 022406. URL: https://link.aps.org/doi/10.1103/PhysRevA.102.022406.

[9] Aleks Kissinger and John van de Wetering. "Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions". In: *Quantum Science and Technology* 7.4 (July 2022), p. 044001. URL: http://dx.doi.org/10.1088/2058-9565/ac5d20.

[10] Dave Wecker and Krysta M. Svore. *LIQUi|>: A Software Design Architecture and Domain-Specific Language for Quantum Computing*. 2014. arXiv: 1402.4467 [quant-ph]. URL: https://arxiv.org/abs/1402.4467.

[11] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. 1998. arXiv: quant-ph/9807006 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9807006.

[12] Sergey Bravyi, Graeme Smith, and John A. Smolin. "Trading Classical and Quantum Computational Resources". In: *Physical Review X* 6.2 (June 2016). URL: http://dx.doi.org/10.1103/PhysRevX.6.021043.

[13] Sergey Bravyi et al. "Simulation of quantum circuits by low-rank stabilizer decompositions". In: *Quantum* 3 (Sept. 2019), p. 181. URL: http://dx.doi.org/10.22331/q-2019-09-02-181.

[14] Sergey Bravyi and David Gosset. "Improved Classical Simulation of Quantum Circuits Dominated by Clifford Gates". In: *Physical Review Letters* 116.25 (June 2016). URL: http://dx.doi.org/10.1103/PhysRevLett.116.250501.

[15] Aleks Kissinger, John van de Wetering, and Renaud Vilmart. "Classical Simulation of Quantum Circuits with Partial and Graphical Stabiliser Decompositions". en. In: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TQC.2022.5.

[16] Hammam Qassim, Hakop Pashayan, and David Gosset. "Improved upper bounds on the stabilizer rank of magic states". In: *Quantum* 5 (Dec. 2021), p. 606. URL: http://dx.doi.org/10.22331/q-2021-12-20-606.

[17] Julien Codsi. "Cutting-Edge Graphical Stabiliser Decompositions for Classical Simulation of Quantum Circuits". MA thesis. University of Oxford, 2022.

[18] Matthew Sutcliffe and Aleks Kissinger. *Procedurally Optimised ZX-Diagram Cutting for Efficient T-Decomposition in Classical Simulation*. 2024. arXiv: 2403.10964 [quant-ph]. URL: https://arxiv.org/abs/2403.10964.